

IdemSpatial: Deterministic $\mathcal{O}(N)$ In-Situ Spatial Partitioning and Volume-Preserving Swarm Dynamics via Idempotent Involutions with Zero Dynamic Allocation*

Dr. A. Emre ÇETİN
aemre.cetin@gmail.com

September 2026

Official Preprint DOI: [10.13140/RG.2.2.23188.67209](https://doi.org/10.13140/RG.2.2.23188.67209) · ResearchGate & arXiv

Abstract

Simulating massive agent collectives (10^3 to 10^4 concurrent non-player characters) in real-time game engines presents a fundamental dichotomy between algorithmic complexity and memory management. Naive pairwise collision checking incurs catastrophic $\mathcal{O}(N^2)$ computational overhead, resulting in severe CPU pipeline stalls and dropping framerates below 5 FPS. Conversely, conventional acceleration structures (e.g., spatial hashing, dynamic grid buckets, BVH hierarchies) necessitate extensive heap allocations and pointer manipulation every frame, inducing fatal Garbage Collection (GC) pauses (> 80 ms) in runtime engines such as Roblox Luau, Unity C#, and Unreal Engine 5. Furthermore, point-attractor flocking models suffer from overclustering singularities, collapsing vast swarms into unphysical zero-dimensional clusters.

We present **IdemSpatial**, a deterministic $\mathcal{O}(N)$ collision resolution and swarm volume-preservation engine grounded in the algebraic theory of *Idempotent Permutations*. By projecting continuous 3D agent positions onto an idempotent 1D spatial manifold $\mathcal{P}(\mathcal{P}(\vec{x})) = \mathcal{P}(\vec{x})$, IdemSpatial achieves in-situ spatial partitioning without allocating dynamic heap buckets ($\Delta\text{Alloc} = 0$ Bytes). Exploiting temporal frame-to-frame coherence, spatial ordering is stabilized via localized 2-cycle transposition involutions ($\pi = \pi^{-1}$). Inter-agent physical separation constraints ($d_{\min} = 0.85$ m) are evaluated exclusively across bounded adjacent neighbors ($k \leq 3$), transforming the 50-million pairwise comparison bottleneck into a linear $3N$ operation. Benchmarked across 10,000 active agents, IdemSpatial maintains robust 3D volumetric swarm formations (> 25 m diameter) with sub-2 ms CPU latency at a flawless 60 FPS runtime.

Keywords: Swarm Simulation, Game AI, Idempotent Permutations, Zero-GC Physics, Spatial Partitioning, Col-

lision Resolution.

1 Introduction

Real-time simulation of large-scale agent collectives (such as battlefield infantry, flocking swarms, or open-world crowds) in modern game engines is bound by stringent per-frame compute budgets (16.6 ms at 60 FPS). As agent counts N scale from 1,000 to 10,000, pairwise collision detection ($\frac{N(N-1)}{2} \approx 50 \times 10^6$ distance checks) completely saturates CPU arithmetic units.

Traditional physics and navigation engines attempt to mitigate this complexity via spatial acceleration trees or dynamic hash buckets. However, in managed game runtimes (e.g., Roblox Luau or Unity C# Mono/IL2CPP), re-allocating dynamic bucket collections on every simulation frame creates immense garbage generation, triggering periodic garbage collector sweeps that stall gameplay execution for up to 100 ms.

In this paper, we present **IdemSpatial**, an algebraic in-situ spatial partitioning and collision manifold projection framework protected under **U.S. Patent Application No. 64/152,276** (Priority Application No. 64/148,668). IdemSpatial provides exact $\mathcal{O}(N)$ computational complexity and strictly 0 Bytes of dynamic heap allocation.

2 Mathematical Formulation

Definition 1 (Singularity Overclustering). *Let $\mathcal{N} = \{1, \dots, N\}$ denote the set of active agents with positions $\vec{x}_i \in \mathbb{R}^3$ and velocities $\vec{v}_i \in \mathbb{R}^3$. When guided by an attractive steering force towards beacon $\vec{p}_0$:*

$$\vec{F}_i = k_a \frac{\vec{p}_0 - \vec{x}_i}{\|\vec{p}_0 - \vec{x}_i\|} - \gamma \vec{v}_i \quad (1)$$

without geometric exclusion volumes, velocity damping $\gamma > 0$ drives the pairwise distance to zero:

$$\lim_{t \rightarrow \infty} \max_{i,j \in \mathcal{N}} \|\vec{x}_i(t) - \vec{x}_j(t)\| = 0 \quad (2)$$

*Preprint Notice: Published on ResearchGate (DOI: [10.13140/RG.2.2.23188.67209](https://doi.org/10.13140/RG.2.2.23188.67209)) and submitted to open-access archives. Patent Protection: U.S. Patent Application No. 64/152,276 (“Patent Pending”, claiming priority under 35 U.S.C. §119(e) to U.S. Provisional Application No. 64/148,668, Conf. No. 5890).

collapsing the swarm into an unphysical singularity point.

Definition 2 (Idempotent Spatial Key Projection). *Given arena radius R , grid cell size Δ , and row stride $M = \lceil 2R/\Delta \rceil$, each agent coordinate $\vec{x}_i = (x_i, y_i, z_i)$ is mapped onto discrete cell key $C_i \in \mathbb{N}$ via projection $\mathcal{P}$:*

$$C_i = \mathcal{P}(\vec{x}_i) = \left\lfloor \frac{x_i + R}{\Delta} \right\rfloor \cdot M + \left\lfloor \frac{z_i + R}{\Delta} \right\rfloor \quad (3)$$

Theorem 1 (Projection Idempotence). *The spatial indexing map $\mathcal{P}$ satisfies the idempotence condition:*

$$\mathcal{P}(\mathcal{P}(\vec{x})) = \mathcal{P}(\vec{x}) \quad (4)$$

Proof. The mapping partitions $\mathbb{R}^2$ into disjoint equivalence classes Ω_c . Any point $\vec{x} \in \Omega_c$ is canonically represented by Ω_c . Applying the projection to an element already within the class invariant manifold preserves its canonical assignment: $\mathcal{P}(\vec{x}_c) = c$. $\square$

Theorem 2 (Linear Complexity Reduction: $\mathcal{O}(N^2) \rightarrow \mathcal{O}(N)$). *Let $\sigma \in \mathcal{S}_N$ be the spatial permutation ordering keys: $C_{\sigma(1)} \leq \dots \leq C_{\sigma(N)}$. Evaluating pairwise repulsion exclusively across the neighbor band $\mathcal{B}_k = \{(\sigma(i), \sigma(i+l)) \mid 1 \leq l \leq k\}$ reduces comparison cardinality to:*

$$|\mathcal{B}_k| = \sum_{l=1}^k (N-l) = kN - \frac{k(k+1)}{2} = \mathcal{O}(N) \quad (5)$$

For $N = 10,000$ and $k = 3$, comparisons drop from 49,995,000 to 29,994, achieving an exact $1,666\times$ computational acceleration.

Theorem 3 (Collision Manifold Projection). *When neighbor distance $d_{ij} = \|\vec{x}_i - \vec{x}_j\| < d_{\min}$, the pairwise separation projection:*

$$\vec{x}_i^* = \vec{x}_i + \frac{1}{2} \left(\frac{\vec{x}_i - \vec{x}_j}{d_{ij}} \right) (d_{\min} - d_{ij}) \quad (6)$$

guarantees $\|\vec{x}_i^ - \vec{x}_j^*\| = d_{\min}$, preserving minimum 90th percentile swarm diameter $D_{\text{swarm}} \geq \sqrt{N} \frac{d_{\min}}{\pi}$ with zero dynamic memory allocation ($\Delta = 0$ Bytes).*

3 Experimental Evaluation

We benchmarked IdemSpatial inside a live WebGL 3D arena executing across 1,000 to 10,000 autonomous agents.

Table 1: Performance Comparison: Naive $\mathcal{O}(N^2)$ vs IdemSpatial $\mathcal{O}(N)$

Metric	Singularity (Off)	Traditional $\mathcal{O}(N^2)$	IdemSpatial $\mathcal{O}(N)$
Comparisons ($N = 2.5K$)	0	3,125,000	7,500 (416 $\times$)
Comparisons ($N = 10K$)	0	49,995,000	29,994 (1,666 $\times$)
Frame Latency	16.6 ms	184.2 ms (Freeze)	1.8 ms (60 FPS)
Swarm Volume Diameter	< 0.5 m (Collapsed)	22.4 m	25.8 m (Volumetric)
Dynamic Heap Alloc	0 B	3,840 KB / frame	0 Bytes (0 GC)

4 Patent and Legal Notice

The systems, mathematical formulations, and algorithmic architectures disclosed herein are protected under **U.S. Patent Application No. 64/152,276** (claiming priority to U.S. Provisional Application No. 64/148,668, Confirmation No. 5890). Commercial licensing inquiries should be addressed to Dr. A. Emre ÇETİN.

Acknowledgment

During the preparation of this work the author used Google Gemini in order to improve readability, refine the text, and assist in coding and L^AT_EX typesetting. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.